Semantic Layer Architecture Evaluation

A POC comparing Superset Native Semantic Views and Cube Core, with alternative approaches and cost considerations

Scope

Sample banking star schema (20,000 synthetic transactions) behind Trino 476. Superset 6.0.0 → 7.0.0rc2 and Cube Core v1.8.6 in one Docker Compose stack.

Approaches evaluated

Four reviewed: Apache Ossie, Superset native, Cube Core, MetricFlow + dbt. Two built and tested in the POC: Superset native and Cube Core.

Decision to be made

Which architecture should go into a time-boxed production validation, and what evidence is still missing before we commit?

Research and evidence cut-off: repository state and public documentation as of 25 October 2026

Executive summary

Business problem: the same KPI can be calculated differently in each report and tool. A semantic layer defines each KPI once and lets every report reuse that definition.

VERIFIED IN POC

What the POC verified

- Superset 6.0.0: 4 YAML datasets with 22 metrics imported via /api/v1/dataset/import/, with validation and smoke tests
- Superset 7.0.0rc2: semantic views with read-only structure and no custom-SQL metrics; Net Fees = 97,755.00
- Cube Core: Superset and Power BI read the same Cube view; Ismailia Main = 43,153,022.01 in both
- Both numbers re-computed directly on the warehouse seed data in this review

NOT DEMONSTRATED

What remains unproven

- Production load, concurrency, failover: no tests exist
- Role-based edit restrictions: only an Admin user existed
- Power BI using a governed Cube measure (the demo used Power BI's own SUM)
- Superset semantic layers are prerelease; both providers are custom code we would own

DECISION NEEDED

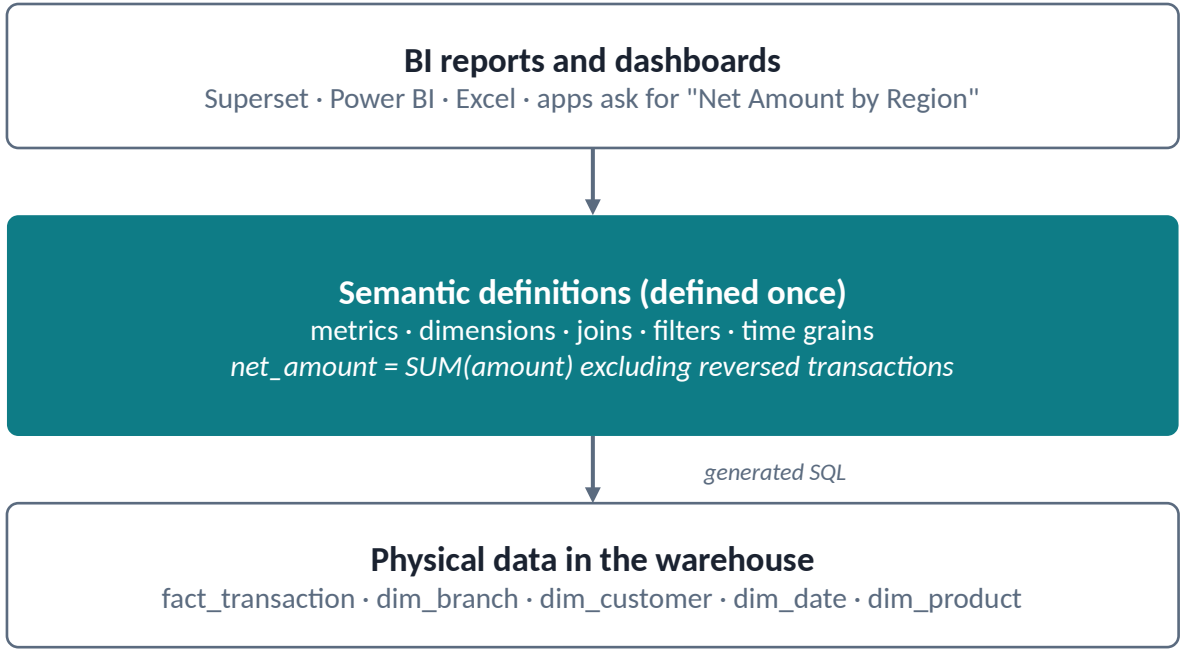
What management must decide

- Is consuming the same KPIs from several tools (Superset, Power BI, apps) a hard requirement within 12–36 months?
- Approve a time-boxed validation phase with explicit pass/fail criteria (slide 20)
- No winner is declared yet: the evidence supports a conditional choice

Core trade-off: Superset native avoids running another service, but its definitions serve Superset only. Cube serves one definition to many tools, but it adds a production service the bank must operate.

Sources: repo semantic/, semantic_views/, cube/model/; .restore/pre-superset-7/superset-metadata.dump; presentation/2.png, 3.1.png, 5.png, 12.png; demo video diagrams/comparison_powerbi_superset.png; [R4] Superset 7.0.0rc2 config.py; [R8] Superset tags.

What is a semantic layer?



Why it matters to the bank

- Consistent KPIs: one agreed formula per number, so management packs agree with each other
- Less duplicated logic: rules such as "exclude reversals" are written once, not in every report
- Reusable definitions: new dashboards start from approved metrics
- Governance: definitions live in Git, are reviewed, tested and deployed like code
- Easier BI change: with an external layer, moving tools does not mean rewriting KPIs

Same data, two reasonable formulas, two answers

SUM(amount)

323,757,585.92

includes 390 reversed transactions

net_amount (governed)

317,418,359.40

reversed transactions excluded

Caveat: a semantic layer guarantees nothing by itself. Tools must actually use the shared definition. In the POC, Power BI showed its own SUM(amount) for Ismailia Main (43,153,022.01), not the governed net_amount (42,507,457.41).

Figures re-computed in this review by loading warehouse/init/01_star_schema.sql (fixed seed) into PostgreSQL 16. Metric definitions: semantic/datasets/Trino/transactions.yaml, cube/model/cubes/transactions.yml. Power BI evidence: presentation/output/professional_demo/QUALITY_REPORT.md.

What does a semantic layer contain?

Component	Plain-language meaning	Example from the POC model
Entities / tables	The business objects the data describes	transactions (fact), branches, customers, dates, products
Relationships / joins	How objects connect; prevents double counting	transactions → branches: many_to_one on branch_key
Dimensions	The ways a number can be sliced	region, channel, segment, product category
Measures / metrics	The agreed formula for a number	net_amount, net_fees, reversal_rate, active_customers
Filters / calculations	Business rules inside a metric; metrics built on metrics	avg_txn_amount = net_amount / non_reversed_count
Time dimensions / grain	Which date drives the metric and how it rolls up	full_date by day, week, month, quarter, year

Terms: a measure is an aggregation (SUM, COUNT DISTINCT). A dimension is an attribute you group or filter by. Grain is the level of detail of one row (here: one transaction). Fan-out is double counting caused by joining to a table with many rows per key.

Excerpt: cube/model/cubes/transactions.yml (line-wrapped)

```
cubes:
  - name: transactions
    sql_table: warehouse.mart.fact_transaction
    joins:
      - name: branches
        sql: "{CUBE}.branch_key = {branches}.branch_key"
        relationship: many_to_one
    dimensions:
      - name: channel
        sql: channel
        type: string
    measures:
      - name: net_amount
        sql: "CASE WHEN NOT {CUBE}.is_reversed
              THEN {CUBE}.amount ELSE 0 END"
        type: sum
      - name: avg_txn_amount
        sql: "{net_amount}
              / NULLIF({non_reversed_count}, 0)"
        type: number
```

Source: repo cube/model/cubes/transactions.yml (Cube Core v1.8.6 model). The excerpt is shortened; the quoted SQL string is wrapped over two lines for the slide.

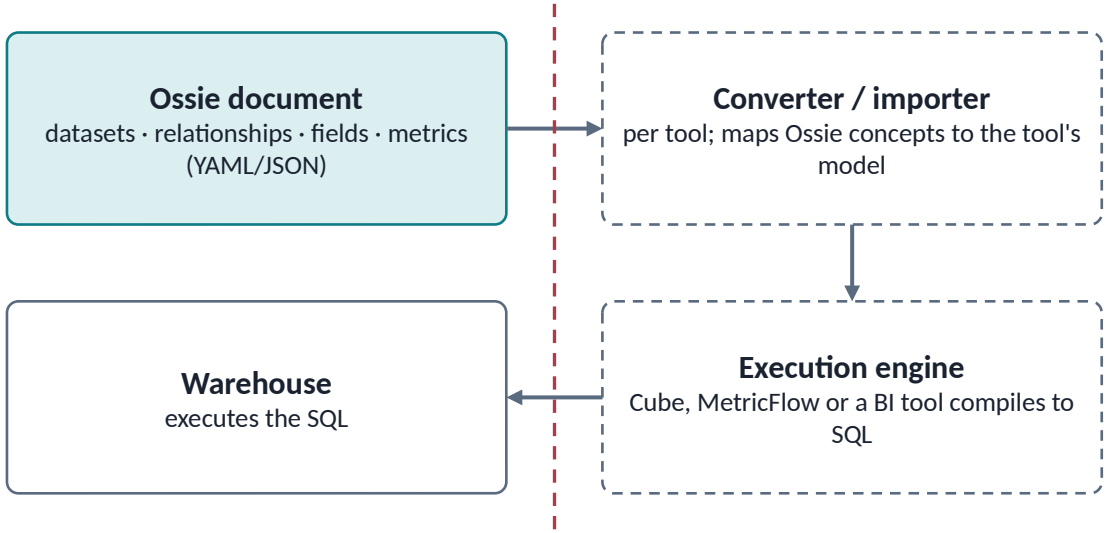
Four approaches — not interchangeable products

	Apache Ossie DOCUMENTED ONLY	Superset native TESTED IN POC	Cube Core TESTED IN POC	MetricFlow + dbt DOCUMENTED ONLY
Definition format	Ossie YAML/JSON spec v0.2.0.dev0 (draft)	YAML in Git → Superset metadata (datasets or layer config)	Cube YAML model: cubes, views, joins	dbt semantic YAML; Ossie input from dbt v1.12
Engine that writes the SQL	None. The consuming tool must translate; a reference engine is a roadmap item	Inside Superset (v6 query builder; v7 provider extension)	Cube API instance: join graph, SQL generation, cache	MetricFlow compiler (Apache 2.0)
Serving API for BI tools	None	Superset charts and SQL Lab only	SQL (Postgres wire), REST, GraphQL	Only in paid dbt platform: JDBC / ADBC / GraphQL
Who can consume it today	Tools with converters or importers (dbt, GoodData, Salesforce, Polaris)	Superset only	Postgres-compatible clients; Power BI via SQL API (limited on Core)	12 listed integrations incl. Power BI, Tableau, Excel; Superset not listed

Provided · Partial / conditional / paid · Not provided — bring your own

Sources: [R22][R24] apache/ossie README, ROADMAP; [R3][R11] Superset 7.0.0rc2 API, Preset SIP-182 blog; [R13][R15][R17] Cube docs; [R28][R29][R30] dbt docs. Superset native column verified in repo code.

Apache Ossie: a format, not an execution engine



Benefits

- Vendor-neutral, Apache-licensed format
- Could reduce lock-in to one tool's model language
- Large partner list; MetricFlow can read it (dbt v1.12+)

Limitations and costs

- Still needs an engine and serving layer: that cost does not disappear
- Converter quality decides fidelity
- Spec is a draft; schema may change

Status at 25 Oct 2026

Item	Status	Classification
Governance	Accepted into the Apache Incubator, June 2026 (formerly Open Semantic Interchange)	Incubating
Specification	core spec v0.2.0.dev0, marked DRAFT	Prerelease
Execution engine	None. "Semantic query language & reference engine" is listed under Future Efforts	Not provided
Superset support	GitHub discussion #42823 (closed, no implementation); Preset proposes a provider extension	Proposed only
Converters	dbt, GoodData, Polaris, Salesforce (reference converters repository)	Officially listed
Snowflake / other platforms	No exact integration confirmed for this deck	Not verified

Risk for the bank: adopting Ossie today means building or waiting for the Superset provider and choosing an engine anyway. Treat it as a future export format, not as an architecture.

Sources: [R22] apache/ossie README; [R23] core-spec/spec.md (0.2.0.dev0 DRAFT); [R24] ROADMAP.md; [R25] DISCLAIMER; [R26] superset discussion #42823; [R11] Preset SIP-182 blog (6 Aug 2026); [R27] Dremio blog; [R28] dbt About MetricFlow.

Option comparison: business pros, cons and cost types

Criterion	Superset native	Cube Core (self-hosted)	MetricFlow + dbt	Ossie + engine
A · Licence	Apache 2.0; no fee	Apache 2.0 / MIT; no fee	Engine free; APIs need paid dbt plan	Spec free; engine cost depends on choice
B · Infrastructure	None beyond Superset	API nodes, Refresh Worker, Cube Store cluster, object storage	dbt runs plus SL service (managed) or a bank-built API	Whatever engine is chosen
C · Warehouse	Every chart query hits Trino	Lower with pre-aggregations, plus refresh jobs	Every metric query compiles to warehouse SQL	Depends on engine
D · Engineering	Low; 2 custom providers to own on v7	Medium: model, Cube Store, auth, CI	High: no dbt project exists yet	High: converters + provider
E · Operations	Superset only	One more tier-1 service: HA, monitoring, upgrades	Vendor-run (managed) or bank-run	Engine-dependent
Cross-BI reuse	Superset only	Many tools via SQL / REST / GraphQL	Power BI, Tableau, Excel listed	Only where converters exist
Governance	Git + CI; v7 locks structure	Git; model rules apply to all clients	Git + dbt CI lifecycle	Format-level only
Maturity	v7 semantic layers prerelease	Stable releases (v1.8.6 tested)	GA platform; Trino unconfirmed	Incubating; draft spec
F · Lock-in / migration	Definitions tied to Superset	Tied to Cube model language	Tied to dbt platform for serving	Lowest in principle

Cost types A–F follow the business cost framework on slide 19. No currency amounts are assigned for self-hosted options; see TCO model. Sources: [R13][R14] Cube production docs; [R29][R31] dbt architecture and pricing; [R4][R8] Superset maturity; [R23][R24] Ossie status; repo superset/extensions/.

POC scope: what was actually built

Approach 1 — Superset native

Git YAML
semantic/ (4 datasets, 22 metrics) · semantic_views/ (2 views)

Sync services (validate · import · smoke test)
sync.py → dataset import API; semantic_views.py → semantic layer API

Superset 6.0.0 → 7.0.0rc2
v6: datasets + saved metrics (Superset writes SQL)
v7: semantic views via custom native provider

Approach 2 — Cube Core (external)

BI clients
Superset (SQL API; v7 Cube provider via REST) · Power BI Desktop (SQL API)

Cube Core v1.8.6
SQL API :15432 · REST :4000
joins + SQL generation
dev mode ON · no Cube Store · no pre-aggregations

cube/model/
5 cubes, 2 views
(Git, not in CI)

Trino 476 shared query engine (catalog: warehouse)

PostgreSQL 16 sample star schema
fact_transaction (20,000 synthetic rows, fixed seed) + 4 dimensions · stand-in for the bank warehouse

Evidence status

VERIFIED IN POC
v6 import: 4 datasets, 22 metrics

VERIFIED IN POC
v7 semantic views, read-only

VERIFIED IN POC
Superset + Power BI on Cube

CONFIGURED
GitLab CI for native only

NOT CONFIGURED
Cube Store, RLS, auth

Not tested in this POC

- Concurrency and load
- Failover / high availability
- Non-admin roles and permissions
- Row-level security
- Real bank data volumes
- Ossie and MetricFlow

Sources: repo docker-compose.yml, sync/sync.py, sync/semantic_views.py, superset/extensions/*, cube/model/*, .gitlab-ci.yml, warehouse/init/01_star_schema.sql; metadata dump .restore/pre-superset-7/superset-metadata.dump; screenshots presentation/2.png, 3.1.png, 5.png, 12.png.

Superset v6: definitions imported as datasets

Definition (YAML in Git)

```
# semantic/datasets/Trino/transactions.yaml (abridged)
table_name: transactions
sql: |-
  SELECT f.transaction_id, d.full_date, b.region, ...
  FROM warehouse.mart.fact_transaction f
  LEFT JOIN warehouse.mart.dim_branch b
         ON f.branch_key = b.branch_key
is_managed_externally: true
metrics:
  - metric_name: net_amount
    expression: SUM(CASE WHEN NOT is_reversed
                    THEN amount ELSE 0 END)
    description: Sum of amounts (EGP), excl. reversed.
```

Request (Superset REST API)

```
# sync/sync.py — import_bundle(), simplified
POST /api/v1/database/import/   (connections first)
POST /api/v1/dataset/import/
  Content-Type: multipart/form-data
  formData  = semantic_layer.zip
              (metadata.yaml, databases/, datasets/)
  overwrite = true

r = s.post(f"{url}/api/v1/dataset/import/",
    files={"formData": ("semantic_layer.zip",
                        zip_bytes, "application/zip")},
    data={"overwrite": "true"})
```

What users could do

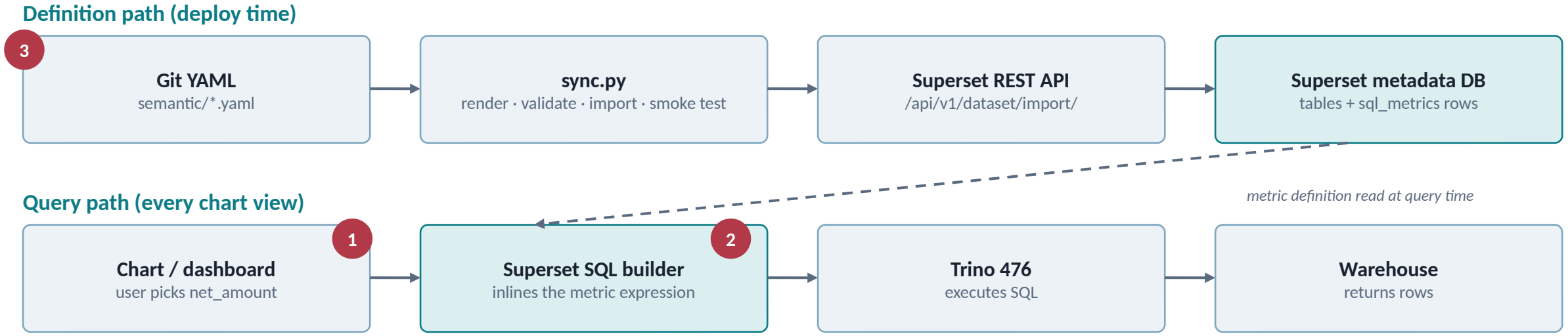
- Pick net_amount, reversal_rate and 20 other metrics by name, with labels and formats
- Build charts on 4 datasets: transactions, fact_transaction, customers, branch_monthly
- Also: write their own ad-hoc (Simple / Custom SQL) metrics in any chart, and use SQL Lab

Defined centrally ≠ locked

- 6.0.0 UI disables Save on is_managed_externally datasets, but the API does not refuse edits (server-side 403 arrives in 7.0).
- Ad-hoc metrics stay open unless disallow_adhoc_metrics is set (not set in the POC).
- Manual edits are overwritten on the next sync. Non-admin roles were not tested.

Sources: repo sync/sync.py, semantic/datasets/Trino/transactions.yaml; [R1] Superset 6.0.0 datasets/api.py (import_); [R2] 6.0.0 DatasourceModal.tsx; [R5] 7.0.0rc2 UPDATING.md. Note: the endpoint is /api/v1/dataset/import/ (singular), not /datasets/import.

Superset v6 data flow — where logic can diverge



Where business logic can diverge

1 Chart authors add ad-hoc SQL metrics next to the governed ones

2 Each virtual dataset repeats rules in SQL: "exclude reversals" appears in transactions, customers and branch_monthly

3 Definitions live only in Superset, so Power BI or Excel must re-implement them

Evidence: metadata backup + smoke-test call

```
-- As stored in Superset 6.0.0 metadata (sql_metrics)
metric_name  | net_amount
table_id     | 4  (transactions)
expression   | SUM(CASE WHEN NOT is_reversed
             |     THEN amount ELSE 0 END)
-- Smoke test per metric (sync.py):
POST /api/v1/chart/data
  {"datasource": {"id": 4, "type": "table"},
   "queries": [{"metrics": ["net_amount"]}]}
```

Sources: repo sync/sync.py (import_bundle, run_metric); semantic/datasets/Trino/*.yaml; .restore/pre-superset-7/superset-metadata.dump (table sql_metrics, row id 9; smoke-test body simplified); [R1] 6.0.0 dataset import API.

Superset v7.0.0rc2: semantic views

semantic_views/*.yaml
source table + joins, dimensions, metrics

↓

semantic_views.py
validate, then upload into the layer config

↓

Semantic layer "Trino"
config stored in Superset (admin-only edits)

↓

Semantic view
read-only structure; saved metrics only

↓

Native provider (custom extension)
compiles each chart query to SQL

↓

Trino → warehouse
executes SQL

API calls (sync/semantic_views.py)

```
# (1) upload definitions
PUT /api/v1/semantic_layer/<uuid>
{"configuration": {
   "default_database": "Trino",
   "views": {"branch_performance":
             {...YAML as JSON...}}}}

# (2) register new views
POST /api/v1/semantic_view/
{"views": [{"name": "branch_performance",
   "semantic_layer_uuid": "<uuid>",
   "configuration": {},
   "description": "Transactions by ..."}]}
```

Definition (branch_performance.yaml, reformatted)

```
name: branch_performance
database: Trino
source:
  table: warehouse.mart.fact_transaction
  alias: t
  joins:
    - table: warehouse.mart.dim_branch
      alias: b
      condition: "t.branch_key = b.branch_key"
metrics:
  - name: net_fees
    expression: SUM(CASE WHEN NOT t.is_reversed
                THEN t.fee_amount ELSE 0 END)
```

Governance: enforced vs still role-dependent

VERIFIED IN POC
UI: "Structure is managed by the upstream semantic layer and is read-only"

VERIFIED IN POC
Explore: "Custom SQL ad-hoc metrics are not enabled"

REPRODUCED
Net Fees = 97,755.00 (0.17 s), matches warehouse

DOCUMENTED ONLY
PUT semantic_view accepts only description, cache_timeout

DOCUMENTED ONLY
Layer edits admin-only; view writes Alpha-tier

NOT DEMONSTRATED
Non-admin roles; SQL Lab still open to DB users

PRERELEASE
Flags SEMANTIC_LAYERS + ENABLE_EXTENSIONS: 'development'

Sources: repo sync/semantic_views.py, semantic_views/branch_performance.yaml, superset/superset_config.py; screenshots presentation/3.1.png, 5.png, 12.png; [R3] semantic_layers/api.py; [R6] schemas.py; [R5] UPDATING.md; [R4][R9] flag lifecycle; [R7] edit modal (all 7.0.0rc2).

Superset v6 vs v7.0.0rc2: why the new API matters

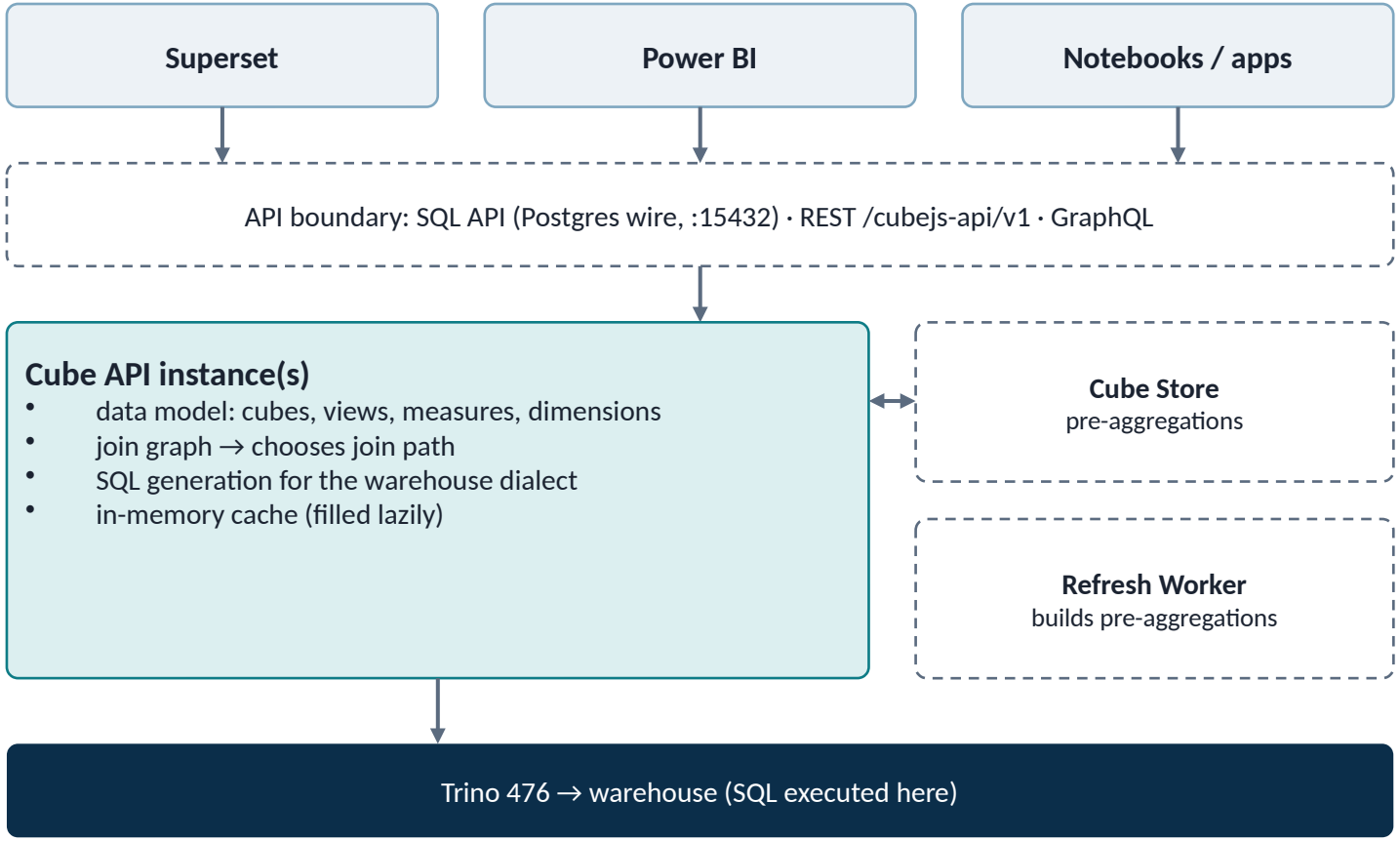
v6: YAML → dataset import → datasets + saved metrics (editable objects) → SQL

v7: YAML → layer config → semantic view (read-only) → provider → SQL

Aspect	v6 — dataset import (6.0.0)	v7 — semantic views (7.0.0rc2)	Evidence
API / model	POST /api/v1/dataset/import/ · SqlaTable datasets	/api/v1/semantic_layer/, /api/v1/semantic_view/ · semantic_view type	Verified
Definition format	Superset export YAML (uuid, columns, metrics)	Own YAML schema of the custom native provider	Verified
Storage / lifecycle	Rows in tables + sql_metrics; sync prunes deleted files	JSON in layer config; views reconciled every 10 s	Verified
Metric editing	UI Save disabled; API open; ad-hoc metrics allowed	Structure read-only; custom SQL metrics disabled	Verified (UI, Admin)
Joins	Hand-written SQL per virtual dataset	Declared per view (source.joins); all joins applied	Code review
Governance	Depends on roles + sync overwrite	Stronger by design; layer edits admin-only	Documented; roles not tested
BI compatibility	Superset only	Superset only (Cube layer reachable as provider)	Verified
Migration effort	—	Rewrite YAML to new schema; charts must move	Estimated
Release risk	Stable release	Release candidate; flags in 'development'	Documented

Sources: repo sync/sync.py, sync/semantic_views.py, superset/extensions/native-semantic-layer; metadata dump; screenshots 3.1.png, 12.png; [R1][R2] Superset 6.0.0; [R3][R4][R5][R6] Superset 7.0.0rc2; [R8] tags; [R10] ASF release policy on release candidates.

Cube Core: architecture and operating model



Dashed = production component not configured in the POC

- Deploy: Docker image cubejs/cube; model files mounted read-only
- Monitor: /readyz health check; production needs auth on, dev mode off

Where the business logic lives

Superset v6
Superset metadata DB (metric SQL) + virtual-dataset SQL with hand-written joins

Superset v7 native
Layer config in Superset; joins listed per view in YAML and always all applied; no join graph

Cube
Cube model files; joins declared once with relationship type; Superset holds no formulas, it asks MEASURE(net_amount)

Sources: [R13] Cube Core architecture; [R14] Running in production; [R15] SQL API; repo docker-compose.yml (cube service), cube/model/*, superset/extensions/native-semantic-layer (_from(): every declared join is emitted).

Cube Core: what the POC demonstrated

View exposed to BI tools

```
# cube/model/views/bank_transactions.yml
views:
  - name: bank_transactions
    cubes:
      - join_path: transactions
        includes: [net_amount, reversal_rate,
                   transaction_count, ...]
      - join_path: transactions.branches
        includes: [branch_name, city, region]
```

Connections (credentials redacted)

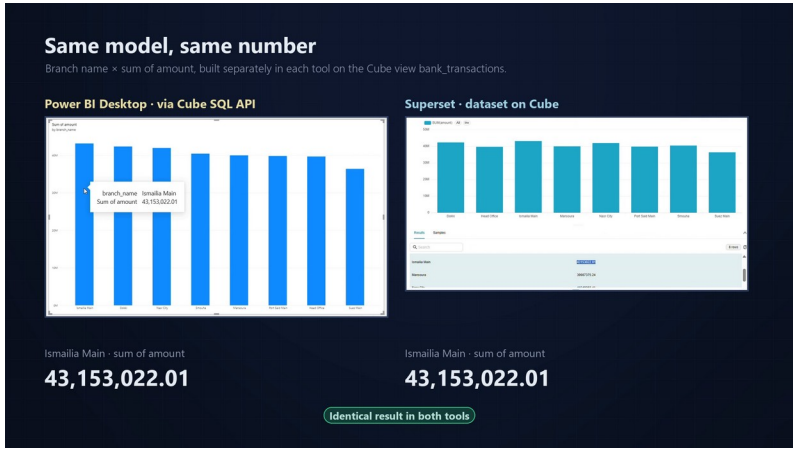
```
# Superset "Cube" connection (bootstrap.sh)
postgresql://cube:<redacted>@cube:15432/cube
# Cube -> warehouse (docker-compose.yml)
CUBEJS_DB_TYPE=trino
CUBEJS_DB_PRESTO_CATALOG=warehouse
CUBEJS_DEV_MODE=true   # POC only
```

Query via SQL API (README)

```
SELECT region,
       MEASURE(net_amount) AS net_amount
FROM bank_transactions
GROUP BY region;
```

Client integration paths

- Superset ↔ SQL API as a Postgres database
- Superset 7 ↔ REST /meta + /load via custom provider
- Power BI Desktop ↔ SQL API (PostgreSQL connector)



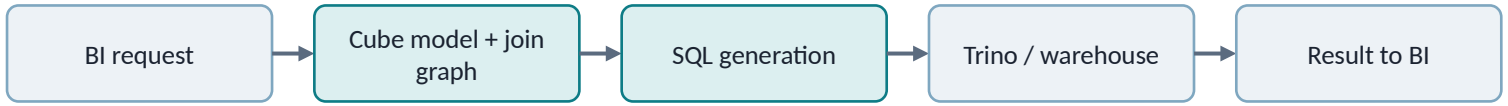
43,153,022.01

Ismailia Main, SUM(amount): same in Power BI and Superset, and re-computed on the warehouse.

SAME VIEW, SAME NUMBER

GOVERNED MEASURE IN POWER BI

Governed net_amount for Ismailia Main = 42,507,457.41



Conceptual flow: Cube's generated SQL text was not captured in the repository

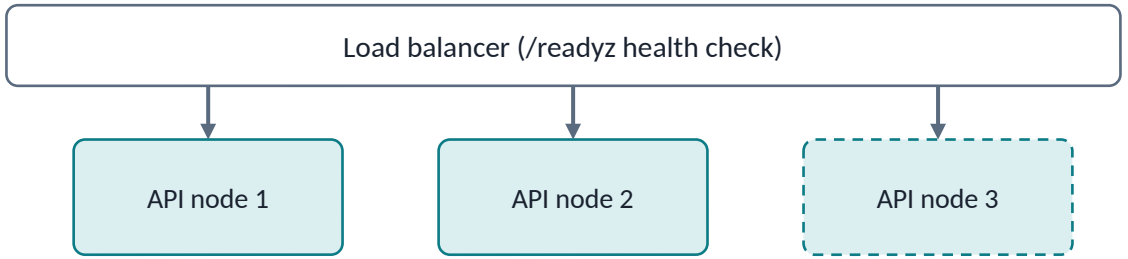
Sources: repo cube/model/views/bank_transactions.yml, superset/bootstrap.sh, docker-compose.yml, README.md; demo still presentation/output/professional_demo/diagrams/comparison_powerbi_superset.png and QUALITY_REPORT.md; [R15][R16][R17][R18] Cube SQL API, MEASURE, Power BI, Superset docs.

Cube Core: cost drivers and scaling

Cost driver	Grows when…
Cube API compute	query rate and concurrent BI users rise
High availability	SLA requires ≥2 API nodes; OSS Cube Store has no replication, so HA = 2 independent clusters
Cube Store + storage	pre-aggregations are enabled; ≥4 CPU, up to 8 GB per node; object storage needed
Refresh jobs (warehouse)	more or fresher pre-aggregations
Warehouse compute	cache misses; un-aggregated queries
Engineering + upgrades	model grows; Cube releases; CI for the model
Integration maintenance	more BI tools or API clients

Sizing is not proven: no load test exists. Cube Cloud's rough guide of 1 API instance per 5–10 requests/second is a reference point, not a bank sizing.

Scaling Cube and scaling the warehouse are separate problems



Scale out: add API nodes for concurrency

Cube Store router + workers
serves pre-aggregations

Trino + warehouse
scale up or out separately

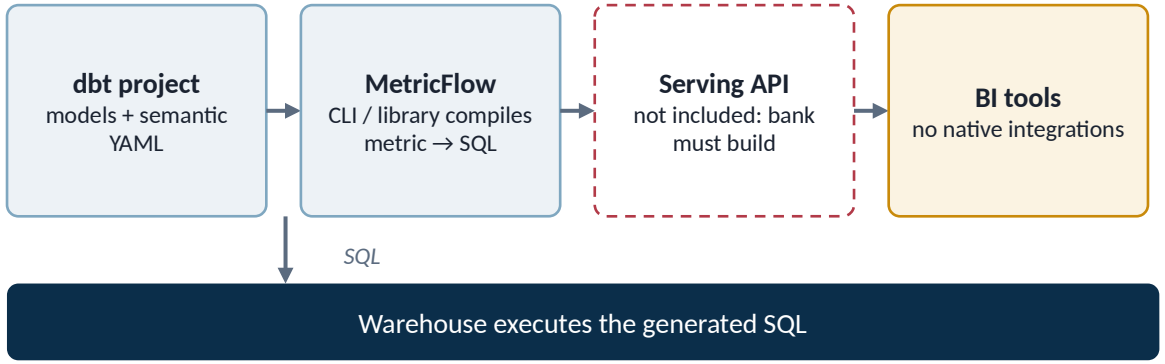
Where is the bottleneck?

Symptom	Likely bottleneck	Lever
Cube CPU high, warehouse idle	Cube API nodes	Add Cube nodes
Warehouse busy, Cube idle	Warehouse / Trino	Pre-aggregate; scale warehouse
Slow only for some charts	Query or model design	Fix model; add rollup
Slow rendering, fast query	BI client	Reduce visuals / rows

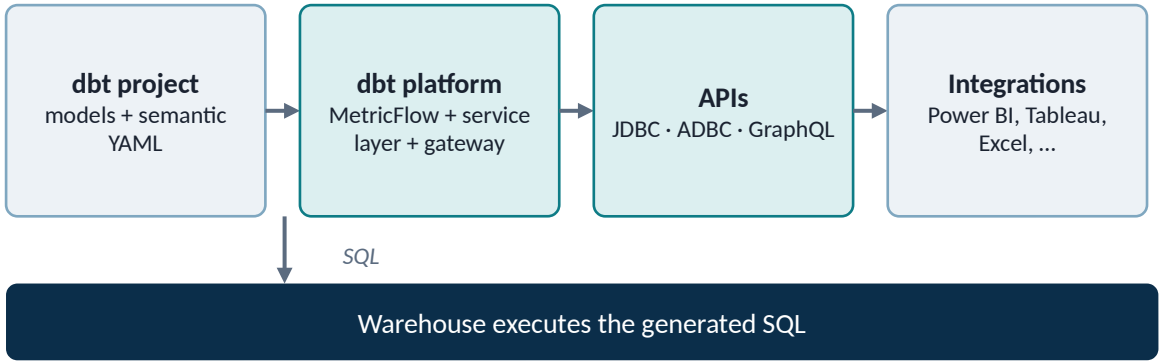
Sources: [R14] Cube Core running in production (Cube Store sizing, no OSS replication, HA via independent clusters, /readyz); [R13] architecture; [R19] Cube Cloud scalability (reference only). POC ran a single Cube container in dev mode without Cube Store.

MetricFlow and dbt: how they work

A · Self-hosted (open-source MetricFlow)



B · Managed dbt Semantic Layer service



Illustrative semantic model (dbt v1.12+ syntax) — not in the POC

```
models:
  - name: fct_transactions   # hypothetical dbt model
    semantic_model:
      enabled: true
    agg_time_dimension: transaction_date
    columns:
      - name: transaction_id
        entity: {type: primary, name: transaction}
      - name: transaction_date
        granularity: day
        dimension: {type: time}
    metrics:
      - name: net_amount
        type: simple
        agg: sum
        expr: "case when not is_reversed then amount else 0 end"
```

What to remember

- dbt builds and tests the tables; MetricFlow turns metric requests into SQL; the warehouse does the work
- The MetricFlow CLI is not an always-on query server: APIs and BI integrations come from the paid dbt platform
- Joins come from entities (primary / foreign keys), avoiding fan-out by design
- Prerequisite: a dbt project. None exists in this repository
- Trino: MetricFlow source has a Trino renderer, but dbt docs do not list Trino. Requires validation

Sources: [R28] dbt About MetricFlow (v1.12 syntax, supported platforms); [R29] Semantic Layer architecture (open vs proprietary components); [R30] integrations; [R33] dbt-labs/metricflow source (SqlEngine.TRINO, sql/render/trino.py). YAML adapted from the dbt docs pattern; field names assume a dbt model that does not exist yet.

MetricFlow: costs, benefits and limitations

Dimension	Self-hosted MetricFlow	Managed dbt Semantic Layer
Licensing / subscription	Apache 2.0: no licence fee	Starter: $100 per user/month, 5 developer seats, 5,000 queried metrics/month. Enterprise / Enterprise+: custom price, 20,000 queried metrics/month
Infrastructure	dbt runs + a bank-built API service, HA and auth	Hosted by dbt Labs; bank provides warehouse connectivity
Warehouse compute	Every metric query runs as SQL on the warehouse	Same; exports can materialise metric tables
Engineering	High: dbt project from scratch + serving layer	Medium-high: dbt project from scratch; integrations configured
API serving / BI	None out of the box; Superset provider not in OSS 7.0.0rc2	JDBC, ADBC, GraphQL; 12 listed integrations; Superset not listed
Vendor dependency	Low (open source)	High: serving tied to dbt platform; data egress / residency review needed
Scaling & reliability	Bank owns capacity planning and on-call	Vendor SLA per contract (not verified)

Benefits

- Metrics governed in the same Git / CI lifecycle as transformations
- Entity-based joins prevent fan-out
- Ratio, derived, cumulative and conversion metric types
- Can read Ossie documents (dbt v1.12+)

Limitations

- No dbt project today: largest upfront effort
- Trino support unconfirmed in docs
- No OSS Superset integration
- Bank-wide price requires vendor quote

Pricing requires vendor confirmation: public list prices (accessed 25 Oct 2026) cannot be extrapolated to a bank-wide Enterprise contract. Overage rates for queried metrics are not published.

Sources: [R31] getdbt.com/pricing (accessed 25 Oct 2026); [R32] How pricing works (queried-metric counting, updated 8 Oct 2026); [R29] SL architecture; [R30] integrations; [R28] About MetricFlow; [R34] MetricFlow Apache 2.0 announcement; Superset 7.0.0rc2 source has no built-in provider [R3].

Decision matrix: when each approach fits

	Superset native	Cube Core	MetricFlow + dbt SL service	Ossie + compatible engine
Best fit	Superset is the only consumer of governed KPIs	Several tools and apps must share one KPI definition	Bank standardises on dbt for transformation	Interchange needed and consumers have verified converters
Main advantage	No extra service; CI already built	Define once, serve many clients; join graph	Metrics in the same lifecycle as models	Vendor-neutral definitions
Main trade-off	Superset-only; v7 prerelease + custom providers	New tier-1 service; shallower Superset UX	Paid platform for serving; no dbt project yet	No engine; draft spec
Primary cost driver	Engineering to maintain providers	Infrastructure + operations	Subscription + build-out	Integration engineering
Scaling	Superset + warehouse	Cube nodes, Cube Store, warehouse	Vendor service + warehouse	Depends on engine
Prerequisites	Stable Superset 7 release; role model	Ops ownership, auth, Cube Store, CI	dbt project; Trino confirmed; contract	Superset provider; engine choice
Reject if	Power BI must show governed KPIs	No team can run it 24/7	Data may not leave the bank network	Production needed within 12 months

Conditional guidance based on the evidence in this deck; not a final recommendation. 'Reject if' conditions are examples to confirm with the business and Risk/IT Security. Sources: slides 6–17 and References appendix.

Total cost of ownership: how to estimate it

This is a cost model, not a measured cost. The repository contains no workload measurements or billing data; no currency amounts are assumed.

Formulas

```
TCO(12|36 mo) = Licensing + Infrastructure + Warehouse
              + Engineering + Operations
              + Integration + Migration

Engineering    = Est. engineering hours
                 × fully loaded hourly cost
Infrastructure = Monthly resource cost × months
Licensing      = Seats × price × months + usage overage
```

Cost categories to fill per option

- Software subscriptions
- Infrastructure (compute, storage, HA)
- Warehouse compute and storage
- Engineering implementation
- Ongoing maintenance and upgrades
- Monitoring and reliability
- Integration development
- Migration
- Testing and release management

- Hours: from validation-phase actuals
- Rates: Finance's fully loaded rate
- Infra: bank's internal VM / K8s price list
- Warehouse: Trino query logs × cost per CPU-hour
- Vendor prices: written quotes only

Sensitivity: which variables move cost most (qualitative)

Variable	Native	Cube	dbt SL
Concurrent users / query rate	M	H	M
Number of BI tools to serve	H*	L	M
Availability target (SLA)	L	H	M
Number of metric developers	L	L	H
Data volume / freshness	M	H	M
Queried metrics per month	—	—	H

H = high, M = medium, L = low cost sensitivity. Judgement based on the cost drivers on slides 7, 15 and 17, not a calculation.

* Native: each extra BI tool re-implements every KPI, so cost scales with tools × metrics.

Full worksheet, assumptions and estimation method: Semantic_Layer_TCO_Model.md. Vendor list prices referenced there: [R20] Cube Cloud, [R31] dbt (accessed 25 Oct 2026). Open-source licensing ≠ zero TCO.

Recommendation: a time-boxed validation before choosing

Decision framework

Must Power BI or other tools show the same governed KPIs within 12–36 months?

YES → validate Cube Core as the shared engine (dbt SL as alternative if dbt is adopted)

NO → keep Superset native: v6 import now, move to semantic views once 7.0 is a final release

In both cases: keep YAML in Git and CI, so a later move costs a conversion, not a rewrite

Validation plan (proposed; scope and dates to be agreed)

1. **Requirements** Confirm which tools must consume governed KPIs, and the SLA
2. **Superset API stability** Re-test semantic views on the first final 7.x release
3. **Same KPIs, both engines** Identical metrics and joins; reconcile to the warehouse
4. **Performance** Representative concurrency and query-latency tests
5. **Warehouse load** Measure Trino CPU and query counts per option
6. **Permissions** Test Gamma/Alpha/custom roles; SQL Lab; Power BI with MEASURE()
7. **TCO** Fill the 12 / 36-month worksheet with measured inputs
8. **Migration & rollback** Document chart migration and the restore procedure

Exit criteria: the chosen option reproduces every governed KPI exactly, meets the agreed latency at target concurrency, enforces roles, and has a signed-off TCO.

Based on findings on slides 2–19. Restore procedure for the pre-upgrade stack exists: .restore/pre-superset-7/RESTORE.md.

Appendix

Evidence log · technical detail · references

Evidence log: claim, proof and status

Claim	Evidence	Status
v6 imported 4 Git-managed datasets and 22 metrics	superset-metadata.dump: tables (is_managed_externally = t), sql_metrics	**Verified**
Add / delete YAML creates / removes a dataset	Demo video segments 10–11 (EDITING_TIMELINE.md)	**Verified**
Superset 6 had no saved charts or dashboards at backup time	superset-metadata.dump: slices = 0, dashboards = 0	**Verified**
Only an Admin user existed	superset-metadata.dump: ab_user_role = 1 row (Admin)	**Verified**
v7 semantic views registered (2 native, 2 Cube)	presentation/2.png (Datasources list)	**Verified**
v7 structure read-only; custom SQL metrics disabled	presentation/3.1.png, 3.2.png, 11.png, 12.png	**Verified**
Net Fees = 97,755.00 on a semantic view	presentation/5.png; re-computed in PostgreSQL 16 from seed SQL	**Reproduced**
Power BI and Superset show 43,153,022.01 via Cube	Demo still + QUALITY_REPORT.md; re-computed from seed SQL	**Reproduced**
Query through the v7 Cube provider returns data	Views listed; no query result captured	**Not demonstrated**
fee_revenue, customer_value native views	Seen in 11.png; YAML not in repository	**Not in repo**
Performance, HA, RLS, non-admin roles	No tests, configs or results in repository	**Not demonstrated**

All file paths relative to the repository root. Seed re-computation: warehouse/init/01_star_schema.sql loaded into PostgreSQL 16 (setseed 0.42) during this review; SQL run directly, not through Superset or Cube.

Technical detail: the custom native provider

Derived from provider code; executed on the seed data

```
-- Query: net_fees by region on branch_performance
-- SQL shape emitted by NativeView._select()
SELECT b.region AS "region",
       SUM(CASE WHEN NOT t.is_reversed
                THEN t.fee_amount ELSE 0 END) AS "net_fees"
FROM warehouse.mart.fact_transaction t
LEFT JOIN warehouse.mart.dim_branch b
       ON t.branch_key = b.branch_key
LEFT JOIN warehouse.mart.dim_date d
       ON t.date_key = d.date_key
GROUP BY 1
LIMIT 10000
```

Region	net_fees
Canal	37,205.00
Greater Cairo	36,100.00
Delta	12,715.00
Alexandria	11,735.00
Total	**97,755.00**

What the bank would own on Superset 7

- Two custom extensions (native-semantic-layer, cube-semantic-layer), ~14 KB of Python each, built on superset_core interfaces
- Every declared join is always emitted, even when unused: no join pruning or fan-out guard
- Not supported by the providers: series limits, custom SQL filters, custom SQL in ORDER BY
- No metric-on-metric composition in the native YAML (Cube supports it)
- Upgrades: the extension API and flags are in 'development' lifecycle and may change
- Security hygiene: test_new_api.py contains a hard-coded admin password; remove before sharing the repo

Sources: repo superset/extensions/native-semantic-layer/dist/backend/src/native_semantic_layer/__init__.py (_from, _select, _predicate), cube_semantic_layer/__init__.py; test_new_api.py (value not reproduced); [R4][R9] lifecycle. Postgres run used schema mart (no Trino catalog prefix).

References & Evidence (1/4)

ID	Claim / topic	Source, version / date	Direct URL
R1	Superset 6.0.0 dataset import endpoint (POST /api/v1/dataset/import/, multipart formData, overwrite, passwords)	apache/superset @ tag 6.0.0 — superset/datasets/api.py (import_) — 6.0.0	https://github.com/apache/superset/blob/6.0.0/superset/datasets/api.py
R2	Externally managed datasets: Save disabled in the dataset editor (frontend)	apache/superset @ 6.0.0 — DatasourceModal.tsx — 6.0.0	https://github.com/apache/superset/blob/6.0.0/superset-frontend/src/components/Datasource/DatasourceModal.tsx
R3	Superset 7 semantic layer / semantic view REST API	apache/superset @ 7.0.0rc2 — superset/semantic_layers/api.py — 7.0.0rc2 (tag commit 16 Oct 2026)	https://github.com/apache/superset/blob/7.0.0rc2/superset/semantic_layers/api.py
R4	SEMANTIC_LAYERS and ENABLE_EXTENSIONS flags default False, lifecycle 'development'	apache/superset @ 7.0.0rc2 — superset/config.py — 7.0.0rc2	https://github.com/apache/superset/blob/7.0.0rc2/superset/config.py
R5	Semantic layer permissions; server-side refusal of edits to externally managed entities	apache/superset @ 7.0.0rc2 — UPDATING.md — 7.0.0rc2	https://github.com/apache/superset/blob/7.0.0rc2/UPDATING.md
R6	Semantic view PUT schema (description, cache_timeout only)	apache/superset @ 7.0.0rc2 — superset/semantic_layers/schemas.py — 7.0.0rc2	https://github.com/apache/superset/blob/7.0.0rc2/superset/semantic_layers/schemas.py
R7	Read-only structure message for semantic views	apache/superset @ 7.0.0rc2 — SemanticViewEditModal.tsx — 7.0.0rc2	https://github.com/apache/superset/blob/7.0.0rc2/superset-frontend/src/features/semanticViews/SemanticViewEditModal.tsx
R8	Superset release tags (no 7.0.0 final tag; 6.1.0 latest final)	apache/superset — tags — checked 25 Oct 2026	https://github.com/apache/superset/tags
R9	Feature-flag lifecycle definitions (Development = experimental, may be incomplete or unstable)	Apache Superset docs — Feature Flags — 'Next' docs, accessed 25 Oct 2026	https://superset.apache.org/admin-docs/configuration/feature-flags

URLs checked during this review (web access or tagged source checkout) on 25 Oct 2026. Full table with 'what the source supports': Semantic_Layer_References.md. Repository evidence is cited by file path on each slide.

References & Evidence (2/4)

ID	Claim / topic	Source, version / date	Direct URL
R10	ASF policy on release candidates	ASF Release Policy (apache/www-site source) — accessed 25 Oct 2026	https://apache.googlesource.com/www-site/+show/HEAD/content/legal/release-policy.md
R11	Superset semantic layers (SIP-182), extensions, providers, Ossie direction	Preset blog — Semantic Layers in Apache Superset: SIP-182 and Apache Ossie — 6 Aug 2026	https://preset.io/blog/semantic-layers-in-superset/
R12	Superset CLI import/export of datasources	Apache Superset docs 6.0.0 — Importing and Exporting Datasources — 6.0.0	https://superset.apache.org/user-docs/6.0.0/configuration/importing-exporting-datasources
R13	Cube Core architecture (API instances, Refresh Worker, Cube Store; caches)	Cube docs — Cube Core architecture — accessed 25 Oct 2026	https://docs.cube.dev/cube-core/architecture
R14	Cube Core production requirements	Cube docs — Running in production — accessed 25 Oct 2026	https://docs.cube.dev/cube-core/running-in-production
R15	Cube SQL API (Postgres protocol), CUBEJS_PG_SQL_PORT, limitations	Cube docs — SQL API — accessed 25 Oct 2026	https://docs.cube.dev/reference/core-data-apis/sql-api
R16	MEASURE() and query types in the SQL API	Cube docs — SQL API query format — accessed 25 Oct 2026	https://docs.cube.dev/reference/core-data-apis/sql-api/query-format
R17	Power BI with Cube (DAX API vs SQL API)	Cube docs — Power BI — accessed 25 Oct 2026	https://docs.cube.dev/docs/integrations/power-bi
R18	Superset with Cube	Cube docs — Superset — accessed 25 Oct 2026	https://docs.cube.dev/admin/connect-to-data/visualization-tools/superset

URLs checked during this review (web access or tagged source checkout) on 25 Oct 2026. Full table with 'what the source supports': Semantic_Layer_References.md. Repository evidence is cited by file path on each slide.

References & Evidence (3/4)

ID	Claim / topic	Source, version / date	Direct URL
R19	Cube Cloud scaling guidance (reference only)	Cube docs — Scalability — accessed 25 Oct 2026	https://docs.cube.dev/admin/deployment/scalability
R20	Cube Cloud pricing (reference for the managed alternative)	Cube — Pricing — accessed 25 Oct 2026	https://cube.dev/pricing
R21	Cube Core licence	cube-js/cube — LICENSE — v1.8.6 tree	https://github.com/cube-js/cube/blob/v1.8.6/LICENSE
R22	Apache Ossie: what it is, repository contents, converters	apache/ossie — README — main, 25 Oct 2026	https://github.com/apache/ossie
R23	Ossie spec version and draft status	apache/ossie — core-spec/spec.md — 0.2.0.dev0 (DRAFT)	https://github.com/apache/ossie/blob/main/core-spec/spec.md
R24	Ossie roadmap: query language and reference engine are future efforts	apache/ossie — ROADMAP.md — main, 25 Oct 2026	https://github.com/apache/ossie/blob/main/ROADMAP.md
R25	Ossie incubation status	apache/ossie — DISCLAIMER — main	https://github.com/apache/ossie/blob/main/DISCLAIMER
R26	Ossie support in Superset — community question	apache/superset Discussion #42823 'Apache Ossie Support' — opened 6 Aug 2026, closed	https://github.com/apache/superset/discussions/42823
R27	Ossie accepted into Apache Incubator (June 2026); formerly OSI	Dremio blog — Apache Ossie joins the Apache Incubator — 2026 (secondary source)	https://www.dremio.com/blog/apache-ossie-the-open-semantic-interchange-joins-the-apache-incubator

URLs checked during this review (web access or tagged source checkout) on 25 Oct 2026. Full table with 'what the source supports': Semantic_Layer_References.md. Repository evidence is cited by file path on each slide.

References & Evidence (4/4)

ID	Claim / topic	Source, version / date	Direct URL
R28	MetricFlow concepts, metric types, supported platforms, Ossie input	dbt docs — About MetricFlow — dbt v1.12+ syntax, accessed 25 Oct 2026	https://docs.getdbt.com/docs/build/about-metricflow
R29	dbt Semantic Layer architecture: open vs proprietary components	dbt docs — Semantic Layer architecture — accessed 25 Oct 2026	https://docs.getdbt.com/docs/use-dbt-semantic-layer/sl-architecture
R30	dbt Semantic Layer integrations	dbt docs — Available integrations — accessed 25 Oct 2026	https://docs.getdbt.com/docs/platform-integrations/avail-sl-integrations
R31	dbt platform pricing	dbt Labs — Pricing — accessed 25 Oct 2026	https://www.getdbt.com/pricing
R32	How queried metrics are counted	dbt docs — How pricing works — last updated 8 Oct 2026	https://docs.getdbt.com/docs/platform/billing/how-pricing-works
R33	MetricFlow licence and source	dbt-labs/metricflow — repository — main, 13 Oct 2026	https://github.com/dbt-labs/metricflow
R34	MetricFlow relicensed to Apache 2.0 (Coalesce 2025)	dbt Labs blog — Open source MetricFlow — Coalesce 2025; page edited 18 Dec 2025	https://www.getdbt.com/blog/open-source-metricflow-governed-metrics
R35	Apache Superset licence	apache/superset — LICENSE.txt — 7.0.0rc2	https://github.com/apache/superset/blob/7.0.0rc2/LICENSE.txt

URLs checked during this review (web access or tagged source checkout) on 25 Oct 2026. Full table with 'what the source supports': Semantic_Layer_References.md. Repository evidence is cited by file path on each slide.